Lớp: TTNT2025

BÁO CÁO KẾT QUẢ THỬ NGHIỆM

Sinh viên thực hiện: Nguyễn Phan Nhật Tân

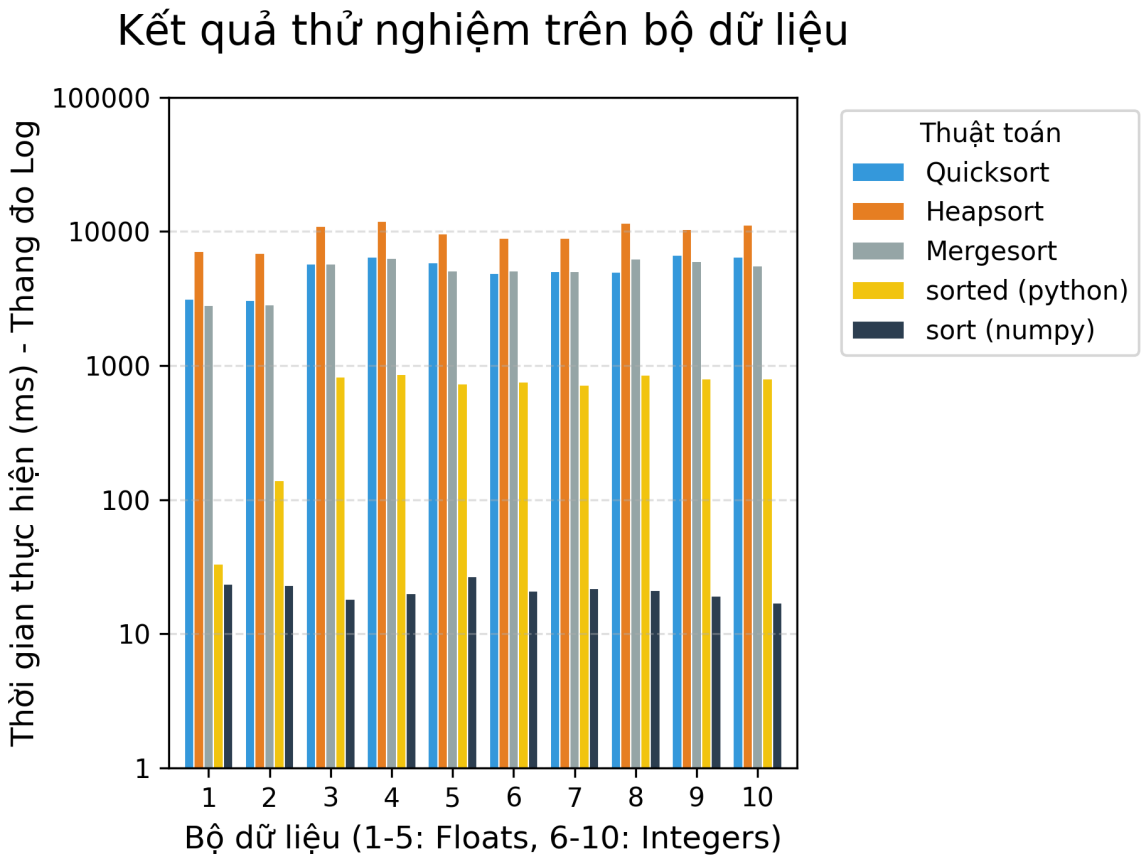
Nội dung báo cáo:

I. Kết quả thử nghiệm

1. Bảng thời gian thực hiện[1]

Dữ liệu	Thời gian thực hiện (ms)				
	Quicksort	Heapsort	Mergesort	sorted (Python)	sort (numpy)
1	3135.4	7089.62	2824.06	32.99	23.4
2	3068.26	6904.76	2854.93	138.67	22.95
3	5712.54	10950.96	5708.89	819.51	18.12
4	6424.02	11979.43	6309.9	859.82	20.04
5	5857.03	9587.68	5107.81	730.03	26.73
6	4888.03	8922.87	5104.57	754.91	20.87
7	5054.6	8951.54	5057.54	718.89	21.83
8	4990.06	11580.8	6252.31	849.71	21.17
9	6632.05	10355.79	6016.03	795.39	19.12
10	6427.8	11215.68	5539.54	800.68	16.99
Trung bình	5218.979	9753.913	5077.558	650.06	21.122

2. Biểu đồ (cột) thời gian thực hiện



[1] Số liệu chỉ mang tính minh họa

II. *Kết luận:*

Xếp hạng theo thời gian thực hiện:

Xếp hạng	Thuật toán	Trung bình (ms)	So với Numpy
1	numpy.sort()	21.12	x1
2	Python sorted()	650.06	x30.8
3	Mergesort	5,077.56	x240.4
4	Quicksort	5,218.98	x247.1
5	Heapsort	9,753.91	x461.9

Phân tích theo dạng dữ liệu

1. Dữ liệu số thực đã được sắp xếp (1 & 2)

Thuật toán	Dataset 1 (Tăng dần)	Dataset 2 (Giảm dần)
Quicksort	3135ms	3068ms
Heapsort	7090ms	6905ms
Mergesort	2824ms	2855ms
sorted()	33ms	139ms
Numpy	23ms	23ms

Nhận xét: Hàm sorted() mặc định của Python (Timsort) tận dụng thứ tự có sẵn cực kỳ hiệu quả. Trên dữ liệu tăng dần (Bộ 1), nó chạy chỉ mất 33ms.

2. Dữ liệu ngẫu nhiên (số thực và số nguyên)

Thuật toán	Trung bình trên số thực	Trung bình trên số nguyên (ms)
Quicksort	5998ms	5598ms
Heapsort	10839ms	10205ms
Mergesort	5709ms	5594ms
sorted()	803ms	784ms
Numpy	21.6ms	20ms

Nhận xét: Tất cả các thuật toán đều thực hiện tốt hơn một chút trên số nguyên so với số thực, có khả năng do chi phí so sánh số nguyên ít tốn tài nguyên hơn.

Phân tích từng thuật toán

- **numpy.sort():** Hiệu năng gần như không đổi trên mọi loại dữ liệu. Khoảng cách khổng lồ (~240-460 lần) so với code tự cài đặt chủ yếu do NumPy chạy ở C-level, loại bỏ hoàn toàn gánh nặng của trình thông dịch Python.

- **Python sorted():** Có sự biến thiên lớn nhất (33ms đến 860ms), cho thấy sự nhạy cảm cực cao với cấu trúc dữ liệu đầu vào.

- **Mergesort:** Về tổng thể thì nhanh hơn Quicksort ~2.7%. Đạt hiệu suất tốt nhất trên dữ liệu sắp xếp sẵn (Bộ 1 & 2).

- **Quicksort:** Việc dùng List Comprehension tạo ra nhiều overhead cho Python mặc dù code nhìn rất gọn.

- **Heapsort:** Chậm gấp gần 2 lần so với Quicksort/Mergesort. Việc gọi hàm heapify() đệ quy hàng triệu lần cộng với việc truy cập bộ nhớ không tuần tự khiến nó không phù hợp để chạy quy mô lớn trong môi trường Python.

Kết luận chính:

- **Trình thông dịch Python là nút thắt cổ chai:** Khoảng cách 240–460× không phải do thuật toán yếu mà là chi phí thực thi bytecode của Python trên từng phép so sánh.

- **Timsort mạnh trên dữ liệu có cấu trúc:** Tốc độ 33ms trên dữ liệu tăng dần chứng minh việc khai thác cấu trúc đầu vào quan trọng ngang với tốc độ thuật toán thô.

- **Mergesort nhỉnh hơn Quicksort trong Python:** Do overhead của list comprehension trong Quicksort nặng hơn cơ chế ghép nối của Mergesort.

- **Heapsort không phù hợp với quy mô lớn trong Python:** Do truy cập bộ nhớ không tuần tự và đệ quy sâu.

- **Median-of-Three hoạt động hiệu quả:** Quicksort trên dữ liệu sắp xếp sẵn nhanh hơn hẳn dữ liệu ngẫu nhiên, xác nhận chiến lược chọn pivot đã tránh được suy biến.

III. Thông tin chi tiết

Link github repo: https://github.com/nguyen-phan-nhat-tan/Sorting-Algorithms-Comparison

1. Báo cáo: report.pdf
2. Mã nguồn: src/benchmark.py
3. Dữ liệu thử nghiệm: src/data